

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Проектирование защищённых интеллектуальных
информационных систем»
на тему:

Анализ безопасности кода.

Часть 1. Анализ безопасности кода разработанного приложения.

Часть 2. Методика оценки безопасности кода

Студенты гр. 321701

Руководитель

И. А. Политыко
В. А. Лукашов
Д. А. Сальников

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Постановка задачи	4
2 Часть 1. Анализ безопасности кода разработанного приложения	5
3 Часть 2. Методика оценки безопасности кода	9
Заключение	12
Список использованных источников	13

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ОЗУ	— Оперативная память;
CRUD	— Create, Read, Update, Delete, Базовые операции над данными;
Fernet	— Симметричная схема шифрования (AES + HMAC);
gcore	— Утилита создания дампа памяти процесса (GDB);
RSS	— Resident Set Size, Резидентный размер процесса в ОЗУ;
HTTP	— Hypertext Transfer Protocol;
PID	— Process Identifier, Идентификатор процесса;
CPU	— Central Processing Unit, Центральный процессор.

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является исследование безопасности программного кода при обработке конфиденциальных и неконфиденциальных данных в оперативной памяти, а также построение методики сравнительной оценки реализаций.

В части 1 необходимо разработать приложение с интерактивным интерфейсом, позволяющее создавать, обновлять и удалять данные непосредственно в ОЗУ. После ввода конфиденциальных значений требуется выполнить их шифрование или хеширование. Далее следует проанализировать защищённость кода, использование оперативной памяти и процессорного времени, снять дампы памяти после создания, обновления и удаления записей и исследовать следы хранения данных.

В части 2 необходимо разработать методику оценки безопасности кода, применить её к собственной реализации и к предоставленному алгоритму, сравнить варианты и сформулировать выводы.

2 ЧАСТЬ 1. АНАЛИЗ БЕЗОПАСНОСТИ КОДА РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ

Краткое описание приложения. Разработано веб-приложение на языке Python с использованием фреймворка Flask. Все пользовательские записи живут только в адресном пространстве процесса и не сохраняются в файловой системе и СУБД. Интерфейс поддерживает создание, обновление и удаление двух классов данных: неконфиденциальных заметок (заголовок и текст хранятся открытым текстом) и конфиденциальных секретов (после ввода тело шифруется симметричной схемой Fernet, в модели остаётся только шифртекст). Ключ шифрования формируется при первом запуске, записывается с правами доступа только для владельца и не встраивается в исходный код. Сервер принимает соединения на локальном интерфейсе.

Интерфейс до и после ввода данных показан на рисунках 2.1 и 2.2.

The screenshot shows the application's main interface titled "Записи" (Records). At the top right, it displays "0 заметок · 0 секретов". The interface is divided into two main sections: "Заметки" (Notes) and "Секреты" (Secrets). The "Заметки" section is labeled "Обычные записи" and contains two input fields: "Заголовок" (Header) and "Текст" (Text), followed by a green "Создать" (Create) button. Below the form, it says "Пока пусто" (Empty for now). The "Секреты" section is labeled "Содержимое хранится в зашифрованном виде" and contains two input fields: "Заголовок" (Header) and "Секрет" (Secret), followed by a green "Создать" (Create) button. Below the form, it also says "Пока пусто".

Рисунок 2.1 — Интерфейс приложения до ввода данных

The screenshot shows the application's main interface after saving a note and a secret. At the top right, it displays "1 заметок · 1 секретов". The "Заметки" section now shows a "Рабочая заметка" (Working note) section with a "Текст обычной записи" (Text of the ordinary record) field, a green "Обновить" (Update) button, and a red "Удалить" (Delete) button. The "Секреты" section now shows a "Рабочий секрет" (Working secret) section with a "Новый текст, если нужно заменить" (New text, if needed to replace) field, a green "Обновить" (Update) button, and a red "Удалить" (Delete) button. The "Создать" (Create) buttons are still present at the bottom of each section.

Рисунок 2.2 — Заметка и секрет после сохранения

Анализ защищённости кода. Безопасный код в смысле методички — это код, который снижает риск нарушения конфиденциальности и доступности при невалидном вводе, сбоях и нагрузке. В разработанном приложении конфиденциальные и неконфиденциальные сущности разделены на уровне типов. Для секрета открытый текст принимается во временный буфер `bytearray`, сразу шифруется, буфер затирается нулями, после чего интерпретатор запрашивает сборку мусора. Обязательные поля и предельные длины проверяются на сервере; сообщения об ошибках не содержат введённого секрета. Журнал приложения не фиксирует тела конфиденциальных записей. На экране списка секретов отображается только заголовок и признак того, что содержимое зашифровано; расшифровка выполняется лишь по явному запросу пользователя и не записывается обратно в хранилище.

```
1 buf = bytearray(plain.encode("utf-8"))
2 try:
3     token = fernet.encrypt(bytes(buf))
4 finally:
5     wipe_bytes(buf)
6     gc.collect()
```

Листинг 2.1 — Шифрование секрета и затирание открытого текста

Ограничения языка остаются существенными. В CPython строки, поступившие из HTTP-формы, могут ещё некоторое время находиться в куче как копии буферов обработки запроса. Поэтому сразу после создания записи маркеры открытого текста иногда обнаруживаются в дампе, даже если в модели секрета уже лежит только шифртекст. Полностью исключить такие копии без нативных расширений нельзя; практическая цель реализации — не удерживать открытый текст в долгоживущих структурах данных.

При аварийном завершении конфиденциальные данные не должны попадать на диск в виде журналов приложения. Дамп процесса, напротив, отражает содержимое ОЗУ и поэтому используется как инструмент анализа, а не как штатный канал хранения.

Ключ Fernet по умолчанию записывается в файл экземпляра с режимом 600. Это упрощает повторный запуск, но противоречит идеалу «конфиденциальное только в ОЗУ»: вместе с дампом процесса ключ с диска позволяет восстановить секреты. Для демонстрации без файла ключ можно передать переменной окружения; на учебном узле файл оставлен, каталог экземпляра закрыт для прочих `uid`.

Анализ оперативной памяти и процессорного времени. Экспериментальный прогон выполнен 12.09.2026. По данным `/proc` и `ps` для работающего сервера: резидентный размер ≈ 37 МБ, виртуальный размер ≈ 128 МБ, один рабочий поток. Профилировщик запускался на стороне

клиента (двадцать запросов состояния и записи): суммарное время около 0,24 с, основная доля — сетевой стек клиента, а не шифрование на сервере. При учебном объёме данных криптография не является узким местом по процессорному времени, а рост памяти определяется прежде всего самим интерпретатором и каркасом веб-сервера.

Анализ дампов оперативной памяти. Дампы процесса снимались после создания, обновления и удаления записей утилитой `gcore`; поиск маркеров выполнялся по текстовым строкам дампа. В качестве неконфиденциального маркера использовалась заранее известная фраза заметки, в качестве конфиденциальных — исходный и обновлённый тексты секрета. Сводка приведена в таблице 2.1.

Сценарий	Заметка	Исходный секрет	Обновлённый секрет	
После создания	1	3	0	
После обновления	2	1	0	
После удаления	1	1	0	

Таблица 2.1 — Число вхождений маркеров в дампах ОЗУ

После создания открытый текст секрета ещё встречается в дампе (кратковременные копии запроса), но не хранится в модели данных. После обновления число вхождений исходного маркера падает, новый открытый текст в дампе не обнаружен. После удаления копия исходного маркера может остаться в буферах интерпретатора, хотя объект секрета из хранилища уже удалён. Неконфиденциальный маркер сохраняется дольше: для заметок это ожидаемо, поскольку они по постановке задачи хранятся открытым текстом. Таким образом, различие классов данных наблюдается не только в исходном коде, но и в фактическом содержимом оперативной памяти.

3 ЧАСТЬ 2. МЕТОДИКА ОЦЕНКИ БЕЗОПАСНОСТИ КОДА

Описание своего и предоставленного алгоритмов. Собственная реализация хранит секреты в ОЗУ только как шифртекст, затирает временный буфер открытого текста и отделяет секреты от обычных замечток на уровне типов и интерфейса.

Предоставленный алгоритм реализует тот же набор операций создания, обновления и удаления, однако поле секрета — обычная строка без шифрования и без очистки буферов. Это типичная ошибка, при которой конфиденциальные данные в куче процесса неотличимы от любых других текстовых полей.

Анализ безопасности предоставленного алгоритма. В предоставленном варианте секрет живёт в поле объекта столько, сколько существует запись. Валидация минимальна: проверяется лишь наличие полей, ограничения длины и затираание отсутствуют. Журнал не содержит тел секретов, однако этого недостаточно: при снятии дампа после создания открытый текст секрета читается как содержимое объекта (маркер BRAVO). После обновления в дампе появляется новый открытый текст (DELTA), после удаления он всё ещё находится. С точки зрения кода нет разделения «хранить открыто / хранить защищённо» и нет криптографической обработки сразу после ввода. В аварийной ситуации и при анализе памяти атакующий с доступом к дампу процесса получает секреты в явном виде. По процессорному времени вариант сопоставим с собственной реализацией, поскольку объём данных невелик; выигрыш по CPU не компенсирует потерю конфиденциальности.

Описание методики. Методика оценивает не внешний вид интерфейса, а то, как код обращается с данными в ОЗУ. Этапы: инвентаризация сущностей и мест хранения; проверка реакции на пустой, чрезмерно длинный и некорректный ввод; анализ времени жизни открытого текста; проверка обновления и удаления по дампам; учёт аварийных сценариев (журнал, дампы процесса); оценка ресурсов; сравнение по чек-листу. Каждый критерий оценивается баллами 0 (нет), 1 (частично), 2 (да). Сумма 0–5 соответствует низкому уровню, 6–11 — среднему, 12–16 — высокому относительно учебных требований работы с памятью.

ID	Критерий
C1	Конфиденциальные данные не хранятся открытым текстом дольше необходимого
C2	После ввода применяется шифрование или необратимое хеширование
C3	Есть затирание временных буферов открытого текста
C4	Конфиденциальные и неконфиденциальные сущности разделены
C5	Валидация обязательных полей и ограничений длины на сервере
C6	Секреты не пишутся в журнал приложения
C7	После удаления маркеры открытого текста секрета не находятся в дампе
C8	Использование памяти и процессорного времени допускает наблюдение и оценку

Таблица 3.1 — Чек-лист методики оценки безопасности кода

Сравнение по методике.

Методика различает варианты по существу лабораторной: предоставленный алгоритм проигрывает по шифрованию, затиранию буферов и следам в дампе; собственная реализация снижает риск чтения секрета из ОЗУ, хотя полностью устранить кратковременные копии строк в интерпретаторе нельзя. Шкала корректна относительно теорбазы работы: критерии покрывают ввод (C5), аварии и журнал (C6), жизнь секрета в ОЗУ (C1–C3, C7), разделение сущностей (C4) и наблюдаемость ресурсов (C8). Балл 2 ставится только при наблюдаемом свойстве, 1 — при частичном (копии HTTP, неполный wipe).

Критерий	Свой	Предоставленный	Комментарий
C1	1	0	у своего возможны краткие копии запроса
C2	2	0	Fernet против открытого текста
C3	2	0	затирание буфера только в своём варианте
C4	2	1	у предоставленного тип секрета есть, семантика открытая
C5	2	1	у своего есть ограничения длины
C6	2	2	оба не журналируют секреты
C7	1	0	после удаления у своего возможна краткая копия; у предоставленного секрет живёт в поле
C8	2	1	оба допускают снятие дампа и замер ресурсов
Итого	14/16	5/16	высокий / низкий уровень

Таблица 3.2 — Сравнение реализаций по чек-листу методики

ЗАКЛЮЧЕНИЕ

В лабораторной работе разработано приложение с операциями над данными в оперативной памяти, выполнен анализ безопасности кода, использования ОЗУ и процессорного времени, проанализированы дампы после создания, обновления и удаления записей, построена методика оценки и проведено сравнение с предоставленным вариантом.

Семантическая составляющая. Безопасность кода определяется не только применением криптографии, но и временем жизни открытого текста в ОЗУ, наличием затирания буферов и явным разделением конфиденциальных и неконфиденциальных сущностей. Дампы памяти делают эти свойства наблюдаемыми: заметки как открытый текст могут оставаться в дампе, тогда как секреты после обработки не должны читаться как исходные строки.

Прагматическая составляющая. Получены воспроизводимый стенд для CRUD в ОЗУ, сценарий снятия дампов и чек-лист методики, позволяющий сравнивать реализации и формулировать улучшения: сокращение копий строк, использование защищённых буферов, контроль журналирования и прав доступа к ключу шифрования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Ховард М., Лебланк Д. Защищённый код. — М. : Русская редакция, 2004. — 704 с.
- [2] Ховард М., Лебланк Д., Вьегга Дж. 24 смертных греха компьютерной безопасности. — СПб. : Питер, 2010. — 400 с.
- [3] Java Virtual Machine Monitoring, Troubleshooting, and Profiling Tool [Электронный ресурс]. — Режим доступа: <http://docs.oracle.com/javase/6/docs/technotes/tools/share/jvisualvm.html>. — Дата доступа: 10.09.2026.
- [4] gcore(1) — Generate a core file of a running program [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man1/gcore.1.html>. — Дата доступа: 10.09.2026.
- [5] Fernet (symmetric encryption) [Электронный ресурс]. — Режим доступа: <https://cryptography.io/en/latest/fernet/>. — Дата доступа: 10.09.2026.